

Lab #9 EGRE246

Modify your lab #7 (repeated below) code to *implement a doubly-linked list* (instead of singly-linked). See the class notes if you're unclear on how this might be done. Re-define and re-implement all of the entities produced for lab #7. In addition add the routine

```
void revPrintList(dllist);
```

that when passed a pointer to a doubly-linked list node prints the list in reverse order. You must perform this operation by traversing the list from the last item to the first one.

Note that if your code for Lab #7 was working correctly, you can just copy over some of your code and it to make it work for doubly-linked lists by adding the extra link.

Below is a start on your code, available on Canvas. Note this will compile as-is and you should just uncomment the appropriate lines as you develop and test your code.

File lab9.c:

```
#include <stdio.h>
#include <stdlib.h>

struct dlnode {

    int data;
    struct dlnode *prev;
    struct dlnode *next;
};

typedef struct dlnode *dllist;

dllist createDList(); // returns empty dllist
void printDList(dllist); // prints dllist
dllist addAtEnd(const dllist l,int n); // adds n to end of l
dllist removeAtFront(dllist);
void revPrintDList(dllist); // prints list in reverse order

int main(void) {
    dllist dl = createDList();
    /*
    printf("newly created list: "); printDList(dl); printf("\n");
    dl = addAtEnd(dl,2);
    dl = addAtEnd(dl,4);
    dl = addAtEnd(dl,6);
    printf("list: "); printDList(dl); printf("\n");
    printf("list reversed: "); revPrintDList(dl); printf("\n");
    */
}
```

```

    dl = removeAtFront(dl);
    printf("after remove front item: "); printDList(dl); printf("\n");
    dl = removeAtFront(dl); dl = removeAtFront(dl);
    dl = removeAtFront(dl); dl = removeAtFront(dl);
    printf("after removing all items: "); printDList(dl); printf("\n");
    */
    return 0;
}

dllist createDList() { return NULL; }

void printDList(dllist dl) {
}

dllist addAtEnd(const dllist l,int n) {
    return NULL;
}

dllist removeAtFront(dllist dl){
    return NULL;
}

void revPrintDList(dllist dl) {
}

```

Lab #7 handout:

Lab #7 EGRE246 Spring 2024

This lab concentrates on learning to create and perform some basic operations on a singly-linked list of integers.

For this project just create a single file with `main` and all of the other needed code.

1. First you need to define the data structure(s) for your program. You will need a `struct` definition for your nodes:

```
struct node {  
    // see notes  
};
```

And define a singly-linked list type to be a pointer to one of these nodes:

```
typedef struct node *llist;
```

2. We will have two kinds of lists to deal with – empty and non-empty lists. We will use the value `NULL` to designate an empty list. And to abstract-away the details on creating a list we will define a very simple function called `createList` that just “returns” an empty list, i.e. `NULL`:

```
llist createList() {  
    return NULL;  
}
```

So add this function to your code. Now we would be able to do the following:

```
llist l = createList();
```

3. Now write a function to print a list, printing the string `NULL` if the list is empty.

```
void printList(llist); // prints contents of list with no newline
```

In writing this function you will have two cases in an if statement and if the list is not empty you traverse the list printing out the contents of each node.

So now the following code should work in `main` and just print `NULL`:

```
llist l = createList();  
printList(l); printf("\n");
```

4. Now add the following function to your code:

```
llist addAtEnd(const llist l,int n); // adds n to the end of list l and
                                   // returns this new list
```

Remember that l can be empty or non-empty so you will need to have two conditions in your function. The general algorithm:

```
create (malloc) a new node
initialize the two fields of this node
if list is empty {
    return this new node (you have a one item list)
} else {
    traverse to the end of the list to get a pointer to the last node
    add this new node to the end (make sure your pointers are all correct)
    return this new list
}
```

So now in main you can do the following:

```
llist l = createList();
printList(l); printf("\n");
l = addAtEnd(l,2);
l = addAtEnd(l,4);
l = addAtEnd(l,6);
printList(l); printf("\n");
```

5. Now add removeAtFront:

```
llist removeAtFront(llist l); // removes first item of l if it exists
```

This one is easier than addAtEnd. The general algorithm:

```
if list l is not empty {
    set a local variable ptr to point to first node of l
    set l to point to the next node
    free the first node
}
return l
```